PBP-ReID 使用说明（中文）

本包是论文《Efficient Body-Prior Token Anchoring and Part-Aware Global–Local Fusion for Multi-Scene Person Re-Identification》的参考实现，在 TransReID / VersReID 的代码框架上改写而来：数据读取、损失、优化器、评测指标沿用原框架，模型部分（ViT-Base + EASA + HPRA + GAL）按论文公式重写。

一、三个模块与代码的对应关系

论文模块	论文位置	作用	代码文件
EASA 身体先验锚定	3.1 节，式 (1)–(3)	无参数，按行把 patch token 切成头/躯干/下肢三个带；不需要关键点或掩码	`model/modules/easa.py`
HPRA 层次部件关系注意力	3.2 节，式 (4)–(8)	第一层：带内自注意力（三个带共享投影，注意力分数条目从 N^2 降到 N^2/P）；第二层：对 3 个带描述子做紧凑注意力	`model/modules/hpra.py`
GAL 全局-局部加性融合	3.3 节，式 (9)–(11)	`F = GAP(X) + W_f · Flatten(X_f) + b_f`，逐元素相加，不扩张通道	`model/modules/gal.py`

损失：BNNeck 特征上的交叉熵（label smoothing）+ 主干特征上的三元组损失（论文 3.4 节）。

二、环境

已验证环境：**Python 3.8 + PyTorch 2.0.0 + CUDA 11.7 + timm 0.9.16**（RTX 3090）。

```
pip install -r requirements.txt
```

`yacs` 可以不装：装了就优先用它，没装会自动回退到 `config/cfg_node.py`（接口一致）。

三、三分钟自检（不需要数据集、不需要预训练权重）

```
python tools/smoke_test.py
```

会依次检查：

1. 模型能构建、参数量打印；
2. EASA 三个带覆盖全部 128 个 token（40/40/48），且 split + merge 能还原原始顺序；
3. 前向/反向正常，梯度能传到所有参数，评测特征数值有限；
4. `F` 与手工计算的 `GAP(X) + W_f · Flatten(X_f) + b_f` 完全一致（式 11）；
5. 表 5 的四种消融配置都能构建并跑通；
6. 一个 197 个位置 token 的 ViT-B 权重能加载进 129 token 的模型，位置编码自动插值，标准块和 HPRA 块的注意力权重都能继承。

全部通过时会打印 `all checks passed on cuda`（或 `cpu`）。

四、跑通完整流程（用合成数据）

```
# 1) 造两个小数据集（Market-1501 风格 + MSMT17 风格）
python tools/make_dummy_dataset.py --layout market --root data/dummy_market
python tools/make_dummy_dataset.py --layout msmt   --root data/dummy_msmt

# 2) 单场景训练 1 个 epoch
python train.py --config_file configs/PBP-ReID.yml \
    DATASETS.ROOTS "['data/dummy_market']" \
    MODEL.PRETRAIN_CHOICE none MODEL.DEVICE_ID "('0')" \
    SOLVER.MAX_EPOCHS 1 SOLVER.IMS_PER_BATCH 8 DATALOADER.NUM_INSTANCE 4 \
    DATALOADER.NUM_WORKERS 0 SOLVER.LOG_PERIOD 1 \
    SOLVER.EVAL_PERIOD 1 SOLVER.CHECKPOINT_PERIOD 1 OUTPUT_DIR logs/smoke

# 3) 用刚存的权重测试
python test.py --config_file configs/PBP-ReID.yml \
    DATASETS.ROOTS "['data/dummy_market']" \
    MODEL.PRETRAIN_CHOICE none MODEL.DEVICE_ID "('0')" \
    TEST.WEIGHT logs/smoke/pbp_reid_1.pth OUTPUT_DIR logs/smoke

# 4) 多场景（两个数据集、统一标签空间）
python train.py --config_file configs/PBP-ReID-multiscene.yml \
    DATASETS.COMBINE_NAMES "['market1501','msmt17']" \
    DATASETS.ROOTS "['data/dummy_market','data/dummy_msmt']" \
    DATASETS.COMBINE_TYPE "[1,2]" \
    MODEL.PRETRAIN_CHOICE none MODEL.DEVICE_ID "('0')" \
    SOLVER.MAX_EPOCHS 1 SOLVER.IMS_PER_BATCH 8 DATALOADER.NUM_INSTANCE 4 \
    DATALOADER.NUM_WORKERS 0 SOLVER.EVAL_PERIOD 1 OUTPUT_DIR logs/smoke_multi
```

合成数据里每个身份的底色都不同，所以 mAP 会直接到 100%。这只说明流程通了，不代表论文精度。

五、真实数据上的训练

1. 下载数据集,修改 `configs/PBP-ReID.yml` 里的 `DATASETS.ROOTS`（单场景）或 `configs/PBP-ReID-multiscene.yml` 里的 7 个路径（多场景）。
2. 把 ViT-Base 预训练权重放到 `ckpts/vit_base_patch16_224.pth`，或改 `MODEL.PRETRAIN_PATH`。
3. 训练：

```
python train.py --config_file configs/PBP-ReID.yml MODEL.DEVICE_ID "('0')" OUTPUT_DIR logs/market1501
```

4. 测试：

```
python test.py --config_file configs/PBP-ReID.yml MODEL.DEVICE_ID "('0')" \
    MODEL.PRETRAIN_CHOICE none TEST.WEIGHT logs/market1501/pbp_reid_60.pth \
    OUTPUT_DIR logs/market1501
```

结果会打印到日志，并写入 `OUTPUT_DIR/test_results.txt`。

PRCC、SYSU-MM01 的评测协议会按数据集名自动切换（`utils/metrics.py`）。

六、复现表 5 的消融（不用改代码）

```
# ViT-Base 参考行：不用 HPRA、不用 GAL
python train.py --config_file configs/PBP-ReID.yml MODEL.HPRA_START 12 MODEL.GAL False
# 只加 HPRA
python train.py --config_file configs/PBP-ReID.yml MODEL.GAL False
# 只加 GAL
python train.py --config_file configs/PBP-ReID.yml MODEL.HPRA_START 12
# 完整模型
python train.py --config_file configs/PBP-ReID.yml
```

- `MODEL.HPRA_START`：从第几个 block 开始用 HPRA（6 表示第 6-11 个，共 6 个；12 表示完全不用）。
- `MODEL.PART_NUM`：带的数量 P，默认 3。
- `MODEL.GAL`：是否启用加性融合。

  注意：EASA 本身无参数，只有被 HPRA/GAL 使用才会改变输出。所以“只加 EASA”在实现上等价于参考行——论文表 5 的 `+EASA only` 与 `reference` 之间的关系，本身就依赖具体实现，这一点在改动实验前需要先想清楚。

七、效率测量

```
python tools/measure_speed.py --batch-size 64 --iterations 100
python tools/count_params_flops.py
```

本机（RTX 3090，单张 256×128，batch 64）实测：

模型	参数量 (M)	主干 (M)	FLOPs(G)	延迟 (ms)	FPS
ViT-B 参考（无 HPRA、无 GAL）	86.32	85.75	22.68	1.54	651
PBP-ReID	**88.10**	**87.53**	**22.46**	**1.57**	**639**
PBP-ReID（第二层独立投影，式 8 字面版）	102.28	101.70	22.46	1.57	636

这三处与论文表 **4** 不一致，投稿/回复审稿意见前务必核对：

1. 参数量：论文写 87.3 M，本实现 88.1 M。差异来自 `W_f` 的形状：代码按式 (10) 用 `PC×C`（1.77 M），若改成三个带共享的 `C×C` 就正好少 0.8 M。要完全对上论文，把 `model/modules/gal.py` 里的投影换成 `C×C` 即可。
2. 第二层投影：式 (8) 写的是独立 `W_Q', W_K', W_V'`（+14.2 M）；论文报的 87.3 M 意味着这组投影与第一层共享，所以代码默认 `MODEL.HPRA_REL_SHARE True`；设成 `False` 才是式 (8) 的字面实现。
3. **FLOPs**：256×128 只有 128 个 patch token，注意力在 ViT-B 一个 block 里只占约 2.7% 的 FLOPs，所以“6 个 block 的注意力省 66%”最多只能省约 1% 的总 FLOPs（22.68 → 22.46 G），达不到论文表 **4** 的 **16%**。论文表 1 说的 −65% 是“注意力分数条目数”（$N^2 \to N^2/P$），这一条代码是严格实现的；而墙钟延迟在此 token 数下与 ViT-B 基本持平。

八、目录结构

```
PBP-ReID/
|--  configs/              单场景 / 多场景配置
|--  config/               defaults.py、cfg_node.py（yacs 缺失时的后备）
|--  model/
|  |--  pbp_reid.py        ViT-Base + EASA + HPRA + GAL
|  |--  make_model.py      BNNeck + 分类器封装
|  |--  vit_utils.py       注意力核、PatchEmbed、位置编码插值
```

```
|  `--  modules/          easa.py、hpra.py、gal.py
|--  datasets/           各数据集读取、采样器、数据增强
|--  loss/ solver/       损失、优化器与学习率
|--  processor/          训练与评测循环
|--  utils/              指标（mAP/CMC/PRCC/SYSU）、日志
|--  train.py test.py
|--  bash/               示例脚本
|--  tools/              自检、效率测量、合成数据
`--  docs/               本文档
```

九、许可与致谢

MIT 许可，见 LICENSE。数据管线、损失、优化器与评测代码改编自 TransReID 与 VersReID，模型部分为本论文实现。使用本代码请同时引用论文与上述两个仓库。